

# Detecção de Domínios Funcionais

## Modelagem de Redes Complexas em Proteínas

### Proteína de Teste

**1A3N**

Hemoglobina humana

4 cadeias (A, B, C, D)

574 resíduos

### Proteína-Alvo

**6B1T**

Adenovírus Humano 5

25 cadeias (A–Y)

~38 000 resíduos

# Pipeline de Execução

Fluxo completo — do arquivo PDB até a validação biológica

1

## Dados PDB

Download 1A3N  
automático  
Upload manual 6B1T  
(2 bundles PDB)

2

## Construção da Rede

Grafo não-dirigido  
Arestas:  $C\alpha-C\alpha \leq 8 \text{ \AA}$   
KDTTree (scipy)

3

## Análise Topológica

Distribuição de graus  
Degree, Closeness,  
Eigenvector,  
PageRank,  
Betweenness

4

## Louvain

Detecção de  
comunidades  
Resolução: 1.0  
Betweenness aprox.  
( $k=300$ ) na 6B1T

5

## Validação

Comunidade × Cadeia  
Comunidade × Família  
RCSB  
ARI, NMI, Purity,  
Homogeneity,  
Completeness

# Instalação e Pré-requisitos

Ambiente: Google Colab (recomendado) · Python ≥ 3.9

## Bibliotecas Python

**biopython** — Parse de arquivos PDB

**networkx** — Construção e análise da rede

**python-louvain** — Detecção de comunidades

**plotly** — Visualização 3D interativa

**pandas / numpy** — Manipulação de dados

**scipy** — KDTree — busca por distância

**scikit-learn** — Métricas de validação (ARI, NMI...)

**matplotlib** — Histogramas e heatmaps

**kaleido** — Exportação de figuras PNG

```
# Célula 1 — instalação
import sys, subprocess
subprocess.check_call([
    sys.executable, "-m",
    "pip", "install", "-q",
    "biopython", "networkx",
    "python-louvain",
    "plotly", "pandas",
    "scipy", "scikit-learn",
    "matplotlib", "kaleido"])
```

## Arquivos PDB necessários:

**1A3N** → download automático:  
(função: `baixar_pdb()`)

**6B1T** → upload manual:

`6b1t-pdb-bundle1.pdb`

`6b1t-pdb-bundle2.pdb`

(extraídos do tar.gz do RCSB)

# Proteína de Teste — 1A3N

Hemoglobina humana · Validação do pipeline antes da execução no alvo

5-6

## Obter & Parsear

`baixar_pdb()` → `carregar_residuos_ca()`

Download automático · extrai Cα de 4 cadeias (A–D)

7

## Construir rede

`construir_rede_por_distancia(df, 8.0)`

574 nós · ~3 490 arestas · exporta CSV

8

## Distribuição de graus

`plot_distribuicao_graus(G_1a3n, ...)`

Histograma + escala log-log → PNG

9

## Centralidades

`calcular_centralidades(G_1a3n, ...)`

Degree · Closeness · Eigenvector · PageRank · Betweenness (exata)

10

## Louvain + validação

`detectar_comunidades_louvain(...)`

14 comunidades · heatmap comunidade × cadeia · ARI calculado

11

## Visualização 3D

`plot_3d_comunidades(...)`

HTML interativo (Plotly) + PNG estático

12

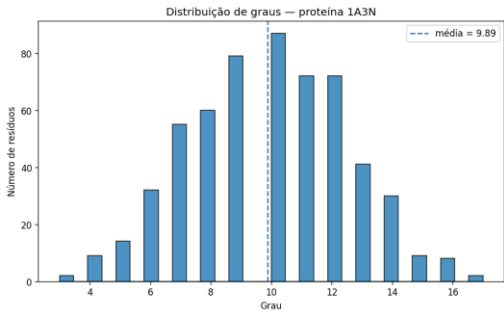
## Exportação

`exportar_resumo_txt(...)`

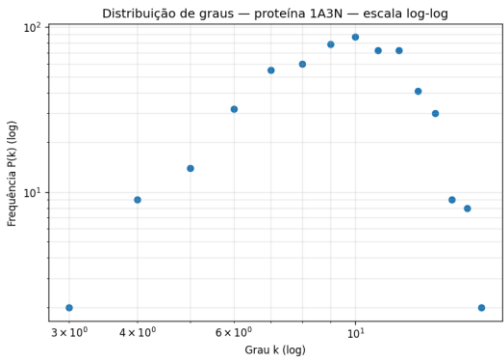
Resumo TXT + CSVs + PNGs em resultados/1A3N/

# Resultados Esperados — 1A3N

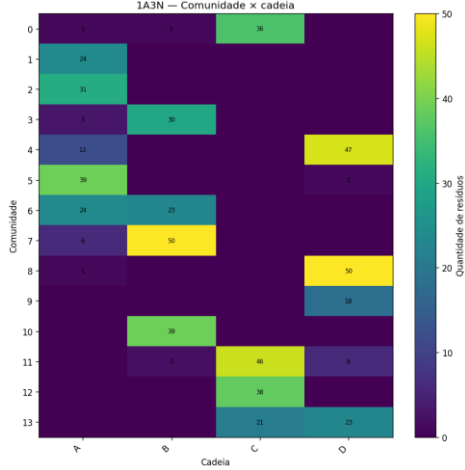
Capturas geradas automaticamente pelo notebook — seções 8, 9 e 10



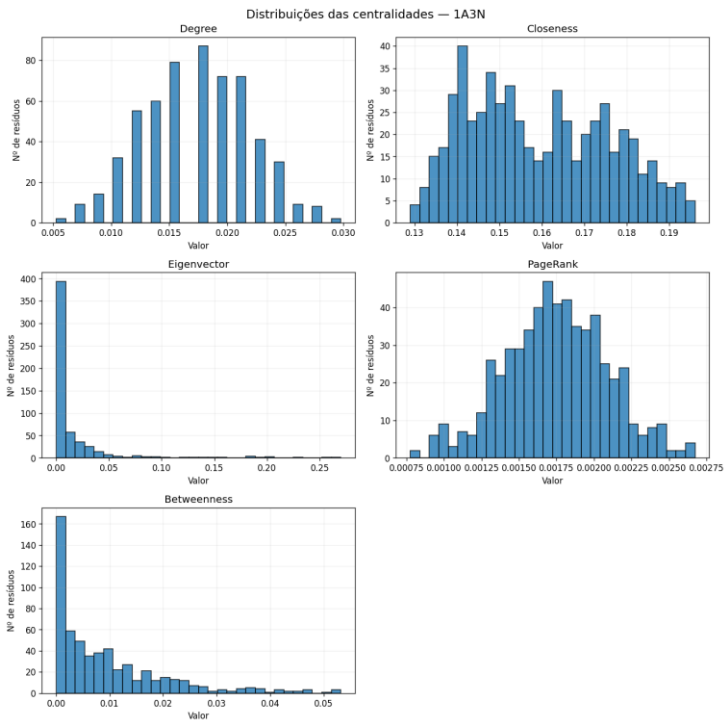
Histograma de graus (média = 9.89)



Escala log-log



Heatmap comunidade x cadeia



Histogramas de centralidades (Degree, Closeness, Eigenvector, PageRank, Betweenness)

# Proteína-Alvo — 6B1T

Adenovírus Humano 5 · 25 cadeias · ~38 000 resíduos · 2 bundles PDB

13

## Upload dos bundles

```
from google.colab import files; files.upload()
```

Upload obrigatório de bundle1.pdb e bundle2.pdb

14

## Parsear resíduos

```
carregar_residuos_ca(pdb_files_6b1t, '6B1T')
```

Lê cadeias A–Y (Hexon, Penton e proteínas auxiliares)

15

## Construir rede

```
construir_rede_por_distancia(df_6b1t, 8.0)
```

~38 000 nós · ~450 000 arestas · pode levar vários minutos

16

## Distribuição de graus

```
plot_distribuicao_graus(G_6b1t, ...)
```

Histograma + log-log · salvos em resultados/6B1T/

17

## Centralidades

```
calcular_centralidades(G_6b1t, k=300)
```

Betweenness aproximada (k=300) — reduz custo computacional

18-19

## Louvain + Cadeia

```
detectar_comunidades_louvain(...)  
pd.crosstab(community, chain_id)
```

Heatmap comunidade × cadeia · modularidade e resumo CSV

20-22

## Validação biológica

```
crosstab(community, familia_rcsb)  
crosstab(community, grupo_minimo)
```

Heatmap × família RCSB · validação mínima Hexon vs Penton

23-26

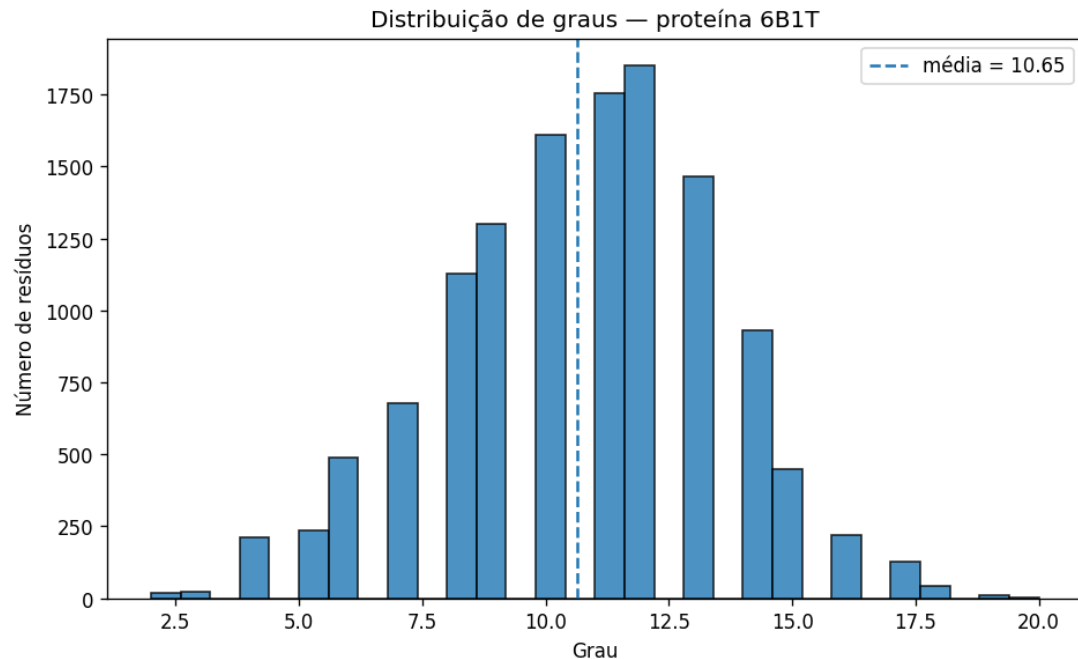
## Métricas + 3D + ZIP

```
adjusted_rand_score / purity / NMI  
plot_3d_comunidades() + make_archive()
```

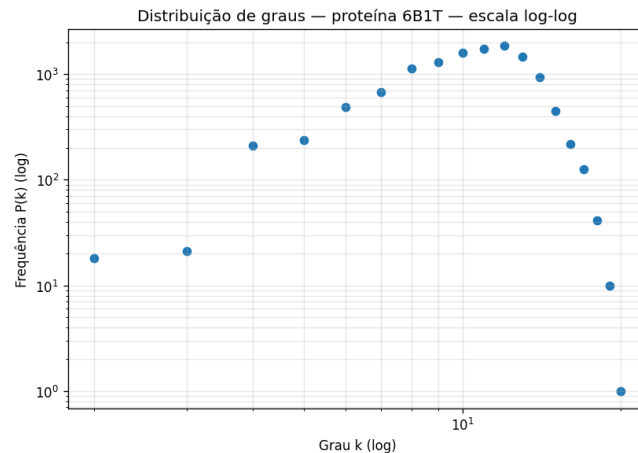
Métricas quantitativas · Plotly 3D · compactação e download

# Resultados Esperados — 6B1T (I)

Distribuição de graus — histograma e escala log-log



Distribuição de graus — 6B1T (média = 10.65)



Escala log-log

~38 000

Nós

~450 000

Arestas

10.65

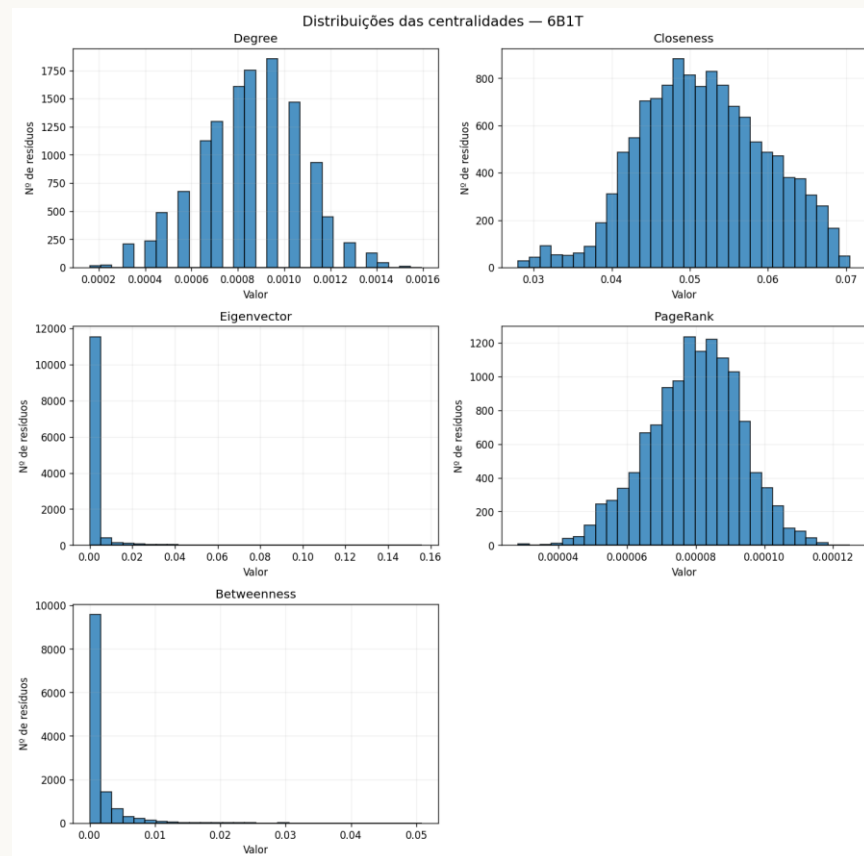
Grau médio

30 (Louvain)

Comunidades

# Resultados Esperados — 6B1T (II)

Distribuições das centralidades — Degree, Closeness, Eigenvector, PageRank, Betweenness

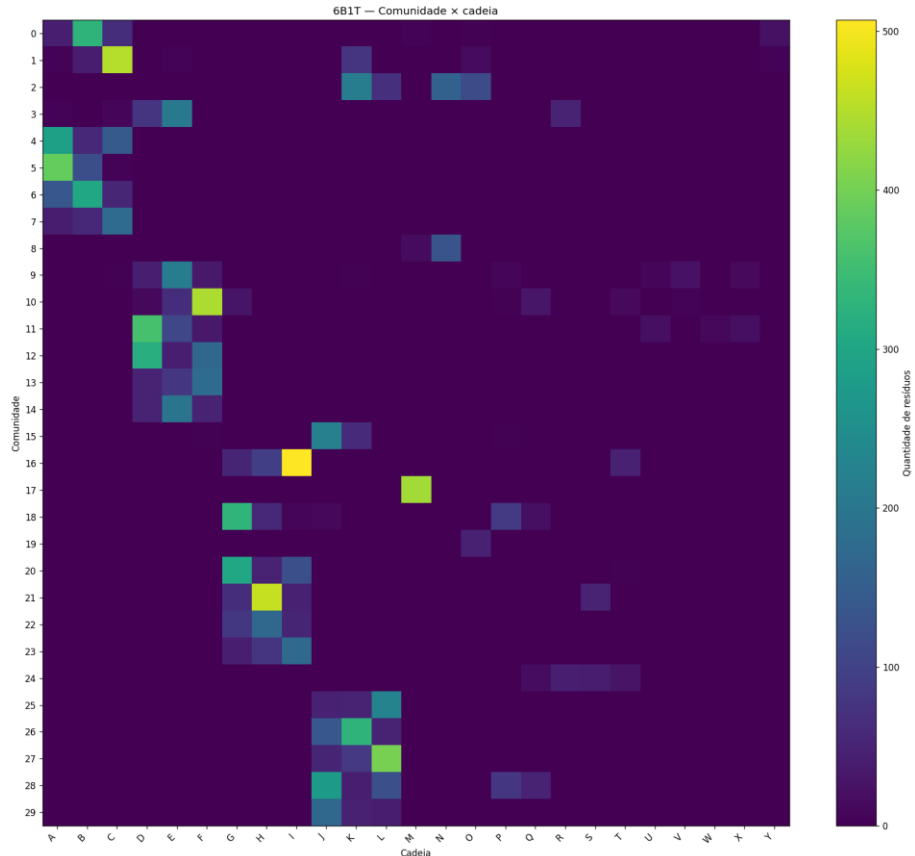


Seção 17 do notebook — Betweenness aproximada ( $k=300$ ) por conta do tamanho da rede



# Resultados Esperados — 6B1T (III)

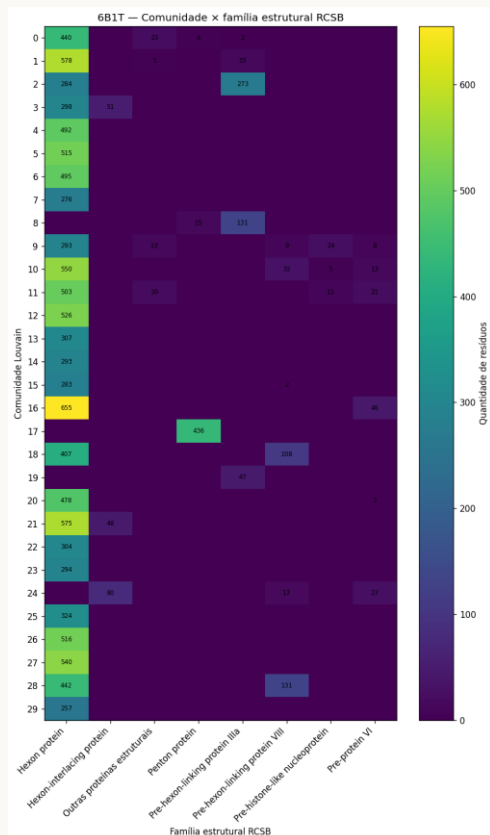
Heatmap comunidade × cadeia — 30 comunidades Louvain × 25 cadeias (A–Y)



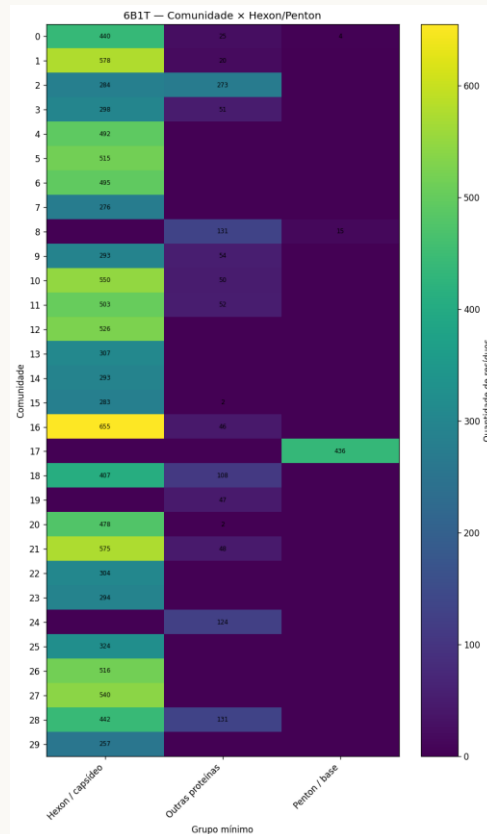
# Resultados Esperados — 6B1T (IV)

Heatmaps de validação biológica — família estrutural RCSB e Hexon/Penton

Comunidade × Família estrutural RCSB



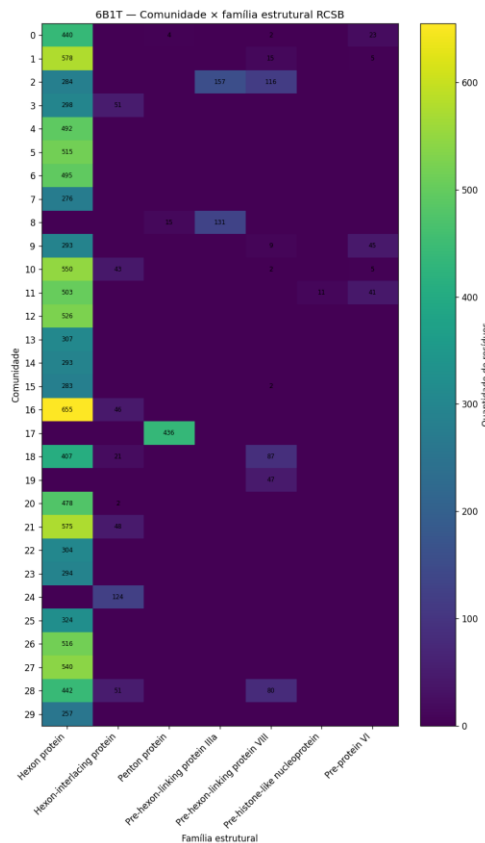
Comunidade × Hexon / Penton



⚠ **Critério mínimo:** Hexon (A–L) e Penton (M) devem pertencer a comunidades dominantes distintas. Verificar em [rcsb.org/3d-view/6B1T](https://rcsb.org/3d-view/6B1T)

# Resultados Esperados — 6B1T (V)

Heatmap comunidade × família estrutural (versão detalhada) e famílias mapeadas



## Famílias Estruturais

**A–L** Hexon protein

**M** Penton protein

**N, O** Pre-hexon-linking IIIa

**P, Q** Pre-hexon-linking VIII

**R, S, T** Hexon-interlacing protein

**U, V, X, Y** Pre-protein VI

**W** Pre-histone-like nucleoprotein

Saídas:  
6B1T\_metricas\_validacao\_biologica.csv

# Repositório & Declarações

## Estrutura do repositório GitHub

### Projeto\_Redes\_Complexas/

```
|— resultados/ # CSVs, PNGs, HTML gerados automaticamente
|— Projeto_Redes_Complexas_Proteina_Teste_1A3N_e_Proteina_Alvo_6B1T.ipynb # pipeline completo 1A3N + 6B1T
|— README.md # instalação · execução · resultados esperados
|— Relatorio_Redes_Complexas_Deteccao_de_Dominios_Funcionais.pdf # relatório final do projeto
|— Slides_Projeto_Redes_Complexas.pdf # apresentação resumida e passo a passo p/ execução do projeto
|— projeto_redes_complexas_proteina_teste_1a3n_e_proteina_alvo_6b1t.py # script Python equivalente ao notebook
```

## Principais saídas do projeto

- ▶ X\_residuos\_ca.csv · X\_arestas.csv
- ▶ X\_metricas\_rede.csv · X\_distribuicao\_graus.csv
- ▶ X\_histograma\_graus.png · X\_graus\_loglog.png
- ▶ X\_centralidades.csv · X\_top20\_<metrica>.csv
- ▶ X\_histogramas\_centralidades.png
- ▶ X\_comunidades\_louvain.csv · X\_resumo\_comunidades.csv
- ▶ X\_heatmap\_comunidade\_x\_cadeia.png
- ▶ 6B1T\_heatmap\_comunidade\_familia\_rcsb.png
- ▶ 6B1T\_metricas\_validacao\_biologica.csv
- ▶ X\_3d\_comunidades.html · resultados\_\*.zip

## Uso de IA

Claude Sonnet 4.6 (Anthropic) — Estrutura dos slides.  
Código revisado e validado pelo aluno.

## Colaborações

Discussões gerais sobre modelagem de redes com colegas da disciplina. Código e relatório de autoria própria.

## Linguagem

Python 3.x · Google Colab · NetworkX, python-louvain, BioPython, Plotly, scikit-learn, scipy, matplotlib.